

An Introduction to Universal Artificial Intelligence

Chapter 2: Background — Section 2.1: Binary Strings

Based on Hutter, Catt & Grau-Moya

CRC Press / Chapman & Hall, 2024

Outline

- 1 Chapter 2 Roadmap
- 2 2.1 Binary Strings: Motivation
 - 2.1.1 Finite Binary Strings and Natural Numbers
 - 2.1.2 Prefix Codes
 - 2.1.3 Infinite Binary Sequences and the Unit Interval
- 3 Summary

Chapter 2: Background — What's in this Chapter? I

Chapter 2 is an *optional* chapter that collects all the prerequisite mathematics used by the rest of the book. If you already know a topic, skip its section; if you find a gap, come back here. Below is a plain-English tour of every section, quoting the epigraph the authors chose:

“Give me six hours to chop down a tree and I will spend the first four sharpening the axe.” — Abraham Lincoln

2.1 Binary Strings (this deck). Defines strings and the operations on them, self-delimiting prefix codes, and the cylinder sets used later to put probability measures on infinite sequences.

Chapter 2: Background — What's in this Chapter? II

2.2 Measure Theory and Probability Theory. States the axioms of probability, defines (semi)measures on infinite sequences, conditional probability, random variables, expectation and variance, key probability inequalities, and convergence of random variables.

2.3 Statistical Inference and Estimation. Introduces the “sunrise problem,” maximum likelihood estimation, and the notion of a consistent estimator — one that learns the truth given enough data.

2.4 Bayesian Probability Theory. Derives Bayes' theorem, Bayesian estimation and prediction, and the classical Laplace rule for predicting binary sequences.

Chapter 2: Background — What's in this Chapter? III

2.5 Information Theory and Coding. Covers Shannon entropy, the Shannon–Fano and arithmetic codes, the Kullback–Leibler divergence, the Kraft inequality, and Shannon's coding theorem — the mathematics of how few bits a message truly needs.

2.6 Computability Theory. Formal models of computation, the Halting Problem, semi-computable functions, and the arithmetic hierarchy — the mathematics of what can and cannot be computed at all.

Chapter 2: Background — What's in this Chapter? IV

2.7 Kolmogorov Complexity. Measures the intrinsic complexity of a string (rather than the randomness of the process that produced it), giving a rigorous formalization of “simplicity” and hence of Occam’s razor. Connects to the Minimum Description Length principle and to Shannon entropy.

2.8 Miscellaneous. Distances between probability distributions (total variation, squared, Kullback–Leibler, Hellinger), how to handle semimeasures (probability that can “leak away”), and how to condition on probability-zero events.

2.9 History and References. Pointers to the literature.

Chapter 2: Background — What's in this Chapter? V

Why start with binary strings? Section 2.1 introduces the fundamental concept of strings and their binary encoding through prefix codes, which allows unique recovery of concatenated and transmitted strings. Every later formal object in the book — programs, data, observations, hypotheses — is ultimately represented as a binary string, so we need to fix this notation precisely before anything else.

2.1 Binary Strings

We assume a brief familiarity with set theory, real analysis, metric spaces, norms, and topological spaces. Often we take these mathematical structures for granted, but to *prove* properties about them later in the book we need a formal definition of every object we use — mathematics provides the tools of rigor that let us do this without endless philosophical discussion.

In this section we define:

- strings and the operations defined on them (2.1.1),
- self-delimiting *prefix codes* (2.1.2),
- and *cylinder sets*, used later to put probability measures on infinite binary sequences (2.1.3).

2.1.1 Finite Binary Strings $\mathbb{B}^*$ and Natural Numbers $\mathbb{N}_0$

We define $\mathbb{B}^n$ as the set of all binary strings of length n (all sequences of n zeros and ones). The set of all finite strings over $\mathbb{B} = \{0, 1\}$ is defined as

$$\mathbb{B}^* := \bigcup_{n=0}^{\infty} \mathbb{B}^n = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, \dots\}.$$

Plain English. $\mathbb{B}^n$ is “all bit patterns of exactly n bits” (e.g. $\mathbb{B}^2 = \{00, 01, 10, 11\}$). Taking the union over every length $n \geq 0$ gives $\mathbb{B}^*$, the set of *all* finite bit strings, including the *empty string* ϵ (length 0, no bits at all).

We typeset bits in monospace to distinguish them from numbers: 101 is the binary string one-zero-one, while 101 is the number “one-hundred-and-one.”

For two strings $x = x_1x_2x_3 \dots x_n$ and $y = y_1y_2y_3 \dots y_m$, the **concatenation** is

$$xy := x_1x_2x_3 \dots x_n y_1y_2y_3 \dots y_m,$$

i.e. writing y right after x . Clearly $\mathbb{B}^*$ is closed under concatenation (gluing two finite strings gives another finite string). The set of *infinite* binary sequences is denoted $\mathbb{B}^\infty$ (studied in 2.1.3).

Notation for Bits, Substrings, and Length

By convention we use *string* or *word* for an element of $\mathbb{B}^*$. For example, $x = x_1x_2x_3 \dots x_n$ is a finite binary string of length n .

- The ***i*th bit** of x is denoted x_i , for $1 \leq i \leq \ell(x)$, where $\ell : \mathbb{B}^* \rightarrow \mathbb{N}_0$ maps a string to its length (number of bits it contains), e.g. $\ell(x_{1:n}) = n$.
- Given i, j with $1 \leq i \leq j \leq n$, the **substring** $x_{i:j}$ (pronounced “ x from i to j ”) is the length- $(j - i + 1)$ segment $x_ix_{i+1} \dots x_j$. For $j < i$ we define $x_{i:j} = \epsilon$ (an empty segment).
- Shorthand: $x_{1:n}$ and $x_{\leq n}$ both mean $x_1x_2x_3 \dots x_n$; $x_{< n}$ means $x_1x_2x_3 \dots x_{n-1}$ (everything strictly before position n).
- Each binary digit is called a **bit**. The empty string ϵ has length $\ell(\epsilon) = 0$.

We also use the standard sets: $\mathbb{R}$ (real numbers), $\mathbb{Q}$ (rationals), and $\mathbb{N}_0 = \{0, 1, 2, \dots\}$ (natural numbers *including* zero).

Interpreting a String as a Number

The standard way to read a binary string as a number in base 2 is

$$b(x) := \sum_{i=1}^{\ell(x)} 2^{\ell(x)-i} \llbracket x_i = 1 \rrbracket$$

Plain English. $b(x)$ “reads off” x as an ordinary binary number: the bracket $\llbracket x_i = 1 \rrbracket$ equals 1 if bit x_i is a 1 and 0 otherwise, and $2^{\ell(x)-i}$ is the place-value (like the “tens,” “hundreds,” etc. columns, but in base 2) of the i th bit. For example, $b(00101) = 5$ (interpreting 00101 in binary), and $b(\epsilon) = 0$ (empty sum).

The catch: b is not a bijection. Leading zeros are invisible to b — e.g. $b(101) = b(0101) = 5$ — so different strings map to the same number, and b cannot be inverted.

The fix. We build a genuine one-to-one correspondence $\langle \cdot \rangle : \mathbb{B}^* \rightarrow \mathbb{N}_0$ between strings and natural numbers by mapping x to the number whose binary representation is $1x$ (prefix a leading 1 to kill the ambiguity), then subtracting 1 so that the empty string maps to 0:

$$\langle x \rangle = b(1x) - 1.$$

The Bijection $\langle \cdot \rangle$: Examples

For example, the string 01 maps to $b(101) - 1 = 5 - 1 = 4$ under this bijection. Some more examples of the bijective mapping $\langle \cdot \rangle$ for strings in $\mathbb{B}^*$:

$\mathbb{B}^*$	ϵ	0	1	00	01	10	11	000	001	...
$\mathbb{N}_0$	0	1	2	3	4	5	6	7	8	...

Plain English. Reading down the table: the strings are enumerated first by increasing length, then lexicographically (0 before 1) within each length — exactly the order you'd count in binary if you started at 0 and never worried about leading zeros.

Proposition 2.1.1: Canonical Bijection from $\mathbb{B}^*$ to $\mathbb{N}_0$

Proposition 2.1.1 (Canonical bijection from $\mathbb{B}^*$ to $\mathbb{N}_0$)

Let x be a binary string. Define

$$\langle x \rangle := 2^{\ell(x)} + b(x) - 1 \quad \text{with} \quad b(x) := \sum_{i=1}^{\ell(x)} 2^{\ell(x)-i} \llbracket x_i = 1 \rrbracket.$$

Then $\langle \cdot \rangle$ is a bijective map, and it enumerates $\mathbb{B}^*$, sorted first by length, then lexicographically.

Plain English. This is the same formula as before, just expanded: since prefixing a 1 to x multiplies every place value by 2 and adds one more $2^{\ell(x)}$ term, $b(1x) = 2^{\ell(x)} + b(x)$, matching $\langle x \rangle = b(1x) - 1$ from the previous slide. *Bijective* means every natural number corresponds to exactly one string, and vice versa — no number is missed, and none is hit twice.

Proof. Left as an exercise to the reader (in the book).

The Inverse Bijection, and Dropping the Notation

The inverse of this bijection can be written as

$$\langle n \rangle^{-1} = b_{\lfloor \log_2(n+1) \rfloor}^{-1}(n + 1 - 2^{\lfloor \log_2(n+1) \rfloor}) \quad (2.1.2)$$

Plain English. $b_k^{-1}(n)$ takes a natural number n and returns a string of k bits representing n in binary with no leading zeros for positive numbers, padded with leading zeros to reach length k (e.g. $b_5^{-1}(7) = 00111$); $b_k^{-1}(n)$ is left undefined if k is too small to fit n 's binary encoding. This ensures the encoded string satisfies $\ell(\langle n \rangle^{-1}) = \lfloor \log_2(n+1) \rfloor$. One can verify this is the valid inverse of $\langle \cdot \rangle$ by rearranging the formula for $\langle x \rangle$ and substituting $\ell(x) = \lfloor \log_2(n+1) \rfloor$.

A notational convenience used throughout the book: once the bijection is fixed, we often *drop* it and let context decide whether a symbol is a string or a number. For instance, we may write $x + 1$ instead of $\langle x \rangle + 1$ when x is a string, or $\ell(n)$ instead of $\ell(\langle n \rangle^{-1})$ for a natural number n .

Proposition 2.1.3: How Big is $\langle x \rangle$?

Proposition 2.1.3 (Bijection length bound)

Let x be a binary string. Then

$$2^{\ell(x)} - 1 \leq \langle x \rangle \leq 2^{\ell(x)+1} - 2.$$

Plain English. The number $\langle x \rangle$ that a length- $\ell(x)$ string maps to is always roughly $2^{\ell(x)}$ — squeezed between $2^{\ell(x)} - 1$ (when x is all zeros) and $2^{\ell(x)+1} - 2$ (when x is all ones). In other words, a string's numeric “address” grows *exponentially* with its length.

Proof. Note that $0 \leq b(x) \leq 2^{\ell(x)} - 1$ (the smallest value of $b(x)$ is 0, all-zero string; the largest is $2^{\ell(x)} - 1$, all-one string), from which the result follows by substituting into $\langle x \rangle = 2^{\ell(x)} + b(x) - 1$. ■

This exponential growth is exactly what makes the “zeroth-order” prefix code inefficient, as we will see in Section 2.1.2.

Python: The Bijection $\langle \cdot \rangle$ Between Strings and Numbers

We implement $b(x)$, the bijection $\langle \cdot \rangle$, and its inverse, and check them against the table of examples above.

```
1 def b(x):
2     """Interpret binary string x as a base-2 number (leading zeros lost)."""
3     return int(x, 2) if x != "" else 0
4
5 def bij(x):
6     """Canonical bijection  $\langle \cdot \rangle$ :  $B^* \rightarrow \mathbb{N}_0$ .  $\langle x \rangle = b(1x) - 1$ ."""
7     return b("1" + x) - 1
8
9 def bij_inv(n):
10    """Inverse of bij:  $\mathbb{N}_0 \rightarrow B^*$ ."""
11    k = (n + 1).bit_length() - 1          # k = floor(log2(n+1))
12    rest = n + 1 - 2 ** k                  # the part after the leading 1
13    return format(rest, "0{}b".format(k)) # k bits, zero-padded
14
15 for s in ["", "0", "1", "00", "01", "10", "11", "000", "001"]:
16     print(f"<{s!r}> = {bij(s)}    inverse: {bij_inv(bij(s))!r}")
```

Output (first few rows): $\langle '' \rangle = 0$, $\langle '0' \rangle = 1$, $\langle '1' \rangle = 2$, $\langle '00' \rangle = 3$, $\langle '01' \rangle = 4$, $\langle '10' \rangle = 5$, $\langle '11' \rangle = 6$, $\langle '000' \rangle = 7$, $\langle '001' \rangle = 8$ — exactly matching the table.

Why We Need Prefix-Free Codes

Often, when transmitting a message across a medium, the message is converted into a sequence of symbols called a **code**, which is then transmitted and decoded to recover the original message. A binary code is ideal since it needs only two distinct symbols — but this is problematic if we want to send *multiple* messages.

The ambiguity problem. Suppose we want to transmit the sequence of numbers $(1, 3)$. We encode each number using the bijection, $\langle 1 \rangle = 0$ and $\langle 3 \rangle = 00$, and transmit the concatenation $\langle 1 \rangle \langle 3 \rangle = 000$. The receiver knows the code, but this is ambiguous: 000 could be decoded as $(3, 1)$, $(1, 1, 1)$, or $(1, 3)$.

A bad fix. We could add a delimiter symbol $\#$ and transmit $0\#00$ — but now we have wasted an extra symbol, which itself needs encoding if we want to transmit *only* binary strings.

The real solution. Restrict the allowed encodings to a **prefix-free** set of binary strings: a set in which no member is a prefix of another. This makes every encoded message *uniquely decodable*, with no delimiter needed.

Definitions: Prefix, Prefix-Free, Prefix Code

Let $x, y \in \mathbb{B}^*$ be strings.

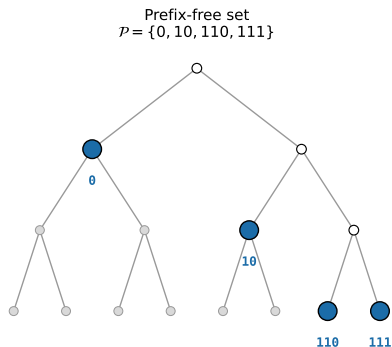
- We say x is a **prefix** of y (denoted $x \sqsubseteq y$) if $\exists z \in \mathbb{B}^*$ such that $xz = y$ (i.e. y starts with x , and the rest is z).
- If additionally $z \neq \epsilon$, we call x a **proper prefix** of y (denoted $x \sqsubset y$) — x is a strictly shorter initial segment of y .
- A set $\mathcal{P} \subseteq \mathbb{B}^*$ is **prefix-free** if no element of the set is a proper prefix of another.
- Given an injection $c : \mathbb{B}^* \rightarrow \mathbb{B}^*$ (a function that never maps two different inputs to the same output), if $\{c(x) : x \in \mathbb{B}^*\}$ is prefix-free, we call c a **prefix-code**. The elements of $\mathcal{P}$ are called **(prefix) codewords**.

Why this matters. Using prefix codes, a message is always *uniquely decodable* — there is no ambiguity in decoding, unlike the $\langle 1 \rangle \langle 3 \rangle$ example above. The message can be transmitted as a sequence of concatenated codewords with *no* reserved delimiter symbol needed to separate the words.

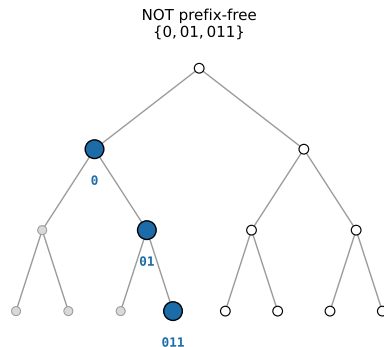
A real-world non-example: Morse code. Morse code $\{ \dots, \text{---} \}$ is *not* a prefix code, as $-.--$ could be decoded as any of $\{Y, KT, NTT, NM, TAT, TW, TEM, TETT\}$. (Morse code actually uses three symbols — a space, plus the absence versus presence of a pulse — and adds spaces between letters as a delimiter.)

Prefix Codes as Binary Trees

Binary tree view of prefix codes (depth-3 strings shown)



No codeword lies on the path to another:
every message is uniquely decodable.



0 is a proper prefix of 01 and of 011:
receiving 011 is ambiguous.

Choosing a codeword “cuts” its branch of the tree, blocking every string below it from also being a codeword.

Reading the Prefix-Code Tree

Plain English. Think of $\mathbb{B}^*$ as a binary tree: the root is ϵ , and each node x has two children, $x0$ and $x1$. Choosing a codeword x “cuts” that branch — every string below x in the tree (every string that has x as a proper prefix) becomes unusable as a codeword, since it would collide with x .

A set is prefix-free exactly when no codeword lies on the tree-path to another, as in $\mathcal{P} = \{0, 10, 110, 111\}$ on the left of the previous figure. The set $\{0, 01, 011\}$ on the right fails: 0 is a proper prefix of both 01 and 011, so seeing 0 arrive does not tell the receiver whether the message is finished or two more bits are coming.

Building an Infinite Family of Prefix Codes

There are (uncountably) many ways to construct prefix codes in $\mathbb{B}^*$. One can construct an infinite family of prefix codes E_i for $\mathbb{B}^*$ by defining, for a string $x \in \mathbb{B}^*$, its **i th-order prefix codeword** recursively as

$$E_i(x) = \begin{cases} 1^x 0 & i = 0 \\ E_{i-1}(\ell(x)) x & \text{otherwise} \end{cases} \quad (2.1.4)$$

where 1^n denotes the string $111 \dots 1$ of n ones (not to be confused with 1 raised to the power of n). We call $E_i(x)$ a **self-delimiting encoding** of x .

Plain English, base case ($i = 0$). $1^x 0$ means: write the digit 1, repeated $\langle x \rangle$ many times (recall we drop $\langle \cdot \rangle$ in the exponent, per the earlier convention), followed by a single 0. The ending 0 acts as a delimiter, so if many zeroth-order codewords are concatenated, we can always tell where one ends and the next begins.

Recursive case ($i \geq 1$). To encode x at order i , first self-delimit its *length* $\ell(x)$ at order $i - 1$, then simply append x itself (no further delimiter is needed, because we now already know how many bits of x to read).

Zeroth-Order Code: Simple but Wasteful

Despite its simplicity, the zeroth-order prefix code E_0 is undesirable: its codewords are much longer than the original message, making it inefficient for practical data transmission. By Proposition 2.1.3,

$$\ell(E_0(x)) = \langle x \rangle + 1 = O(2^{\ell(x)}),$$

which grows **exponentially** with the length of x .

Plain English. Encoding a message of, say, 20 bits this way could require over a *million* bits — clearly undesirable. We would prefer an encoding with length approximately equal to $\ell(x)$, with minimal overhead. (The code cannot map *every* string x to a codeword shorter than $\ell(x)$: that would give an injection from $\mathbb{B}^n$ to $\mathbb{B}^{<n}$, which is impossible by a simple counting argument, since $\mathbb{B}^{<n}$ has fewer than 2^n elements.) Due to this, we often resort to a *higher-order* prefix code.

First- and Second-Order Prefix Codes

We often use the **first-order prefix code** $\bar{x}$ of x :

$$\bar{x} := E_1(x) = E_0(\ell(x))x = 1^{\ell(x)}0x.$$

Plain English. First self-delimit the *length* of x using the (wasteful, but now applied to a much smaller number) zeroth-order code, then append x itself.

And the **second-order prefix code** x' of x :

$$x' := E_2(x) = E_1(\ell(x))x = \overline{\ell(x)}x = 1^{\ell(\ell(x))}0\ell(x)x.$$

Plain English. Self-delimit the length of x using the *first-order* code instead — cheaper still, because now even the wasteful part ($1^{\ell(\ell(x))}0$) is applied to the tiny number $\ell(\ell(x))$.

Footnote (notation subtlety). Strictly, ℓ cannot be composed with itself since $\ell : \mathbb{B}^* \rightarrow \mathbb{N}_0$ takes a string and returns a number, not a string; formally one should write $x' = \overline{\langle \ell(x) \rangle}^{-1} x$. We keep $\langle \cdot \rangle$ implicit throughout, exactly as promised earlier.

Higher-order prefix codes offer diminishing returns and are seldom used.

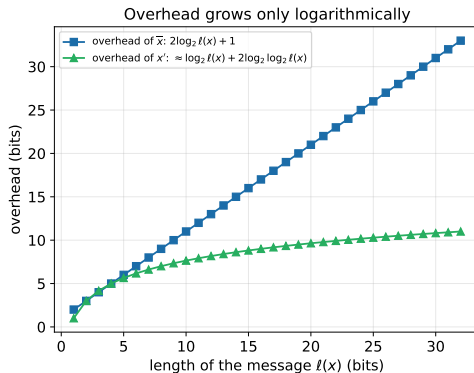
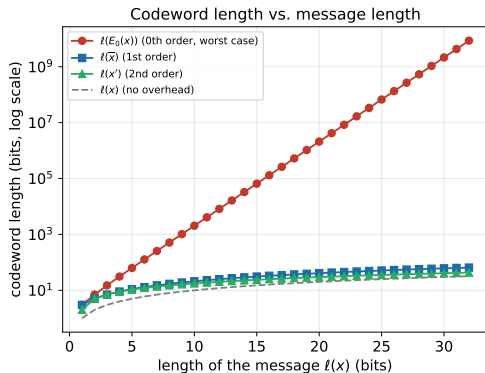
How Much Overhead Do $\bar{x}$ and x' Add?

The lengths of $\bar{x}$ and x' are bounded as follows:

$$\begin{aligned}\ell(\bar{x}) &= 2\ell(x) + 1 \\ \ell(x') &= \ell(x) + 2\ell(\ell(x)) + 1 \\ &= \ell(x) + 2\log_2(\ell(x)) + O(1).\end{aligned}\tag{2.1.5}$$

Plain English. $\bar{x}$ adds at most $2\log_2 \ell(x)$ bits of overhead to the length of x (since $\ell(x) \leq 2^{\ell(\ell(x))}$ by Proposition 2.1.3, so $\ell(\ell(x)) = O(\log_2 \ell(x))$); and x' adds only $\log_2 \ell(x) + 2\log_2 \log_2 \ell(x)$ overhead. Both grow as $O(\log_2 \ell(x))$ — a modest penalty compared to the length of the message itself, unlike the exponential blow-up of E_0 .

Overhead in Pictures: E_0 vs. $\bar{x}$ vs. x'



Left: E_0 grows exponentially in $\ell(x)$ while $\bar{x}$ and x' track $\ell(x)$ closely. Right: the overhead of $\bar{x}$ and x' alone, both $O(\log_2 \ell(x))$.

Lemma 2.1.6 and Theorem 2.1.7

Lemma 2.1.6 (i th-order code is prefix)

$E_i(\cdot)$ is a prefix code for all $i \in \mathbb{N}_0$.

Proof idea. The case $i = 0$ was discussed above (the trailing 0 delimits). For $i = 1$: suppose $\bar{x}$ were a proper prefix of $\bar{y}$ for $x \neq y$, i.e. $\bar{x}z = \bar{y}$ for some $z \neq \epsilon$. Then $1^{\ell(x)}0xz = 1^{\ell(y)}0y$; the strings must agree up to the first zero, forcing $\ell(x) = \ell(y)$; but then $\ell(x) + \ell(z) = \ell(\bar{x}z) = \ell(\bar{y}) = \ell(x)$, forcing $\ell(z) = 0$, i.e. $z = \epsilon$, a contradiction. The proof for $i > 1$ follows by induction.

Theorem 2.1.7 (Uniqueness of prepending prefix codes)

For any two given strings $x, y \in \mathbb{B}^*$ and any prefix code c , $c(x)y$ is uniquely decodable.

Plain English & proof idea. Since c is a prefix code, the prefix of $c(x)y$ that lies in the range of c must be *unique* (because $c(x)$ is in the range of c , we can separate $c(x)y$ into $c(x)$ and y unambiguously). We can then recover x from $c(x)$, and whatever remains is y . This motivates encoding a *pair* of strings (x, y) as $\bar{x}y$ or $x'y$ — leaving the *last* string unencoded saves bits versus encoding both, e.g. $\bar{x}\bar{y}$.

Python: Building Self-Delimiting Codes

We implement E_0 and the first-order code $\bar{x}$ directly from definition (2.1.4).

```
1 def E0(x):
2     """Zeroth-order self-delimiting code: 1^<x> 0."""
3     n = bij(x)                # <x>, the bijection defined earlier
4     return "1" * n + "0"
5
6 def xbar(x):
7     """First-order code: E0(len(x)) x = 1^len(x) 0 x."""
8     return "1" * len(x) + "0" + x
9
10 print("E0('01')    =", E0("01"))      # <01> = 4  -> "11110"
11 print("xbar('101')  =", xbar("101"))    # len=3   -> "1110101"
```

Output: $E_0('01') = 11110$, $\text{xbar}('101') = 1110101$ — matching $\ell(\bar{x}) = 2\ell(x) + 1 = 7$ from (2.1.5).

Python: Verifying Unique Decodability (Theorem 2.1.7)

We now check that concatenating a first-order codeword with an unencoded string, $\overline{x}y$, is uniquely decodable with no delimiter needed.

```
1 def decode_pair(code):
2     """Decode xbar(x) y  ->  (x, y), Theorem 2.1.7 in action."""
3     n = code.index("0")           # position of first 0 = len(x)
4     x = code[n + 1 : n + 1 + n]   # len(x) was unary-coded before the 0
5     y = code[n + 1 + n : ]
6     return x, y
7
8 x, y = "101", "0011"
9 transmitted = xbar(x) + y
10 print("transmitted =", transmitted)
11 print("decoded      =", decode_pair(transmitted), " expected =", (x, y))
```

Output: transmitted = 11101010011, decoded = ('101', '0011') = expected — confirming that $\overline{x}y$ is uniquely decodable with no delimiter between x and y .

Infinite Binary Sequences $\mathbb{B}^\infty$

Theorem 2.1.7 motivates encoding a pair of strings (x, y) as $\bar{x}y$ or $x'y$: leaving the last string unencoded is more economical of bits than encoding *both* as $\bar{x}\bar{y}$ or $x'y'$, and this turns out to be crucial for certain definitions related to Kolmogorov complexity (Section 2.7).

Recall that the set of infinite binary sequences is denoted $\mathbb{B}^\infty$. An element $\omega \in \mathbb{B}^\infty$ is a **one-way infinite sequence**, written $\omega = \omega_{1:\infty} = \omega_1\omega_2\omega_3\dots$ with $\omega_i \in \mathbb{B}$ for all i .

A useful mental picture. There is a correspondence between elements of $\mathbb{B}^\infty$ and predicates (Boolean-valued functions) over $\mathbb{N}^+$. For example, the predicate “ i is a prime number” corresponds to the binary sequence

$$\omega_i = \begin{cases} 1 & i \text{ is prime} \\ 0 & \text{otherwise.} \end{cases}$$

Infinite sequences may also be **prefixed** with a finite string: for $x = x_{1:n}$ and $\omega = \omega_{1:\infty}$, their concatenation is $x\omega = x_1x_2\dots x_n\omega_1\omega_2\dots$.

From Infinite Sequences to Real Numbers

We can draw a correspondence between infinite binary sequences and real numbers. Given $\omega_{1:\infty} \in \mathbb{B}^\infty$, define $f : \mathbb{B}^\infty \rightarrow [0, 1]$ by

$$f(\omega_{1:\infty}) = \sum_{n=1}^{\infty} 2^{-n} \omega_n. \quad (2.1.8)$$

Plain English. This is exactly the map that reads ω as the binary expansion $0.\omega_1\omega_2\omega_3\dots$ of a real number in $[0, 1]$: each bit ω_n contributes 2^{-n} (a half, a quarter, an eighth, and so on) to the sum if it is a 1, and nothing if it is a 0.

Example 2.1.9 (Mapping infinite sequences to real numbers)

Write $(x_{1:n})^\infty$ for the infinite sequence formed by repeating the finite string $x_{1:n}$ forever. Then:

$$f(0^\infty) = 0; \quad f(010^\infty) = 0.25; \quad f(110^\infty) = 0.75; \quad f(10^\infty) = f(01^\infty) = 0.5; \quad f((01)^\infty) = 1/3.$$

The catch: f is surjective but not injective. As shown above, both 10^∞ (i.e. $0.1000\dots$) and 01^∞ (i.e. $0.0111\dots$) map to 0.5 — because real numbers do not have a unique binary expansion (just as $1 = 1.000\dots = 0.999\dots$ in base 10).

Cylinder Sets: Giving $\mathbb{B}^\infty$ a Geometric Picture

We have a geometric representation of $\mathbb{R}$ as the infinite number line. One can ask what geometric representation can be associated with $\mathbb{B}^\infty$ — the answer is **cylinder sets**:

Definition 2.1.10 (Cylinder sets)

The cylinder set Γ_x of a string $x \in \mathbb{B}^*$ is the subset of $\mathbb{B}^\infty$ containing all (one-way) infinite sequences starting with x :

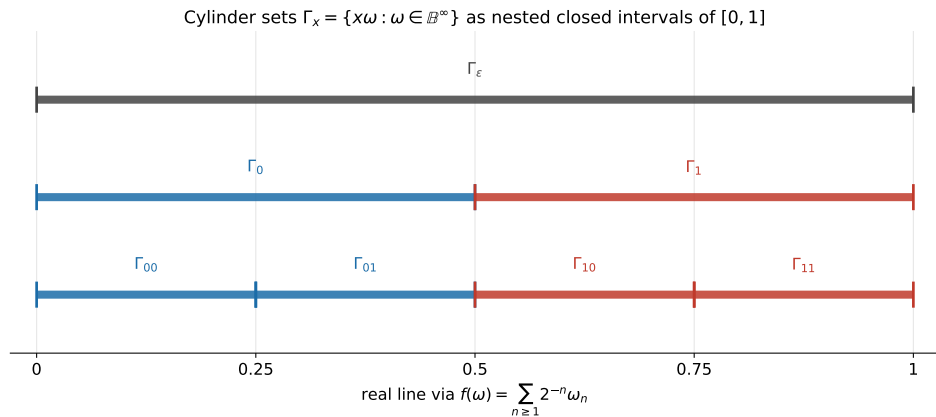
$$\Gamma_x := \{x\omega \mid \omega \in \mathbb{B}^\infty\}.$$

Plain English. Γ_x is “everything that begins with the prefix x ”: fix the first $\ell(x)$ bits to be exactly x , and let the rest of the sequence be arbitrary. For example, Γ_{01} is the set of *all* infinite sequences whose first two bits are 0 then 1.

Remark 2.1.11 (Cylinder sets as intervals)

Geometrically, the cylinder Γ_x can be identified with the closed interval $f(\Gamma_x)$ (using map f from (2.1.8)) of $\mathbb{B}^\infty$ under f .

Cylinder Sets as Nested Intervals of $[0, 1]$



Fixing more bits of the prefix x shrinks the interval $f(\Gamma_x)$ by half each time.

Reading the Cylinder-Set Picture

Plain English. Every cylinder set Γ_x corresponds to a closed sub-interval of $[0, 1]$ of width exactly $2^{-\ell(x)}$: fixing *more* bits of the prefix x shrinks the interval by half each time. Γ_ϵ (the empty prefix, i.e. no constraint at all) is the whole interval $[0, 1]$. Γ_0 is the left half $[0, 0.5]$ and Γ_1 is the right half $[0.5, 1]$; Γ_{00}, Γ_{01} split Γ_0 into quarters, and likewise Γ_{10}, Γ_{11} split Γ_1 .

This nested-interval picture is exactly what will let us put a *probability measure* on $\mathbb{B}^\infty$ in Section 2.2, by assigning probabilities to cylinder sets the same way we would assign lengths to these intervals.

Python: The Map f and Example 2.1.9

We verify Example 2.1.9 numerically. Each infinite sequence is approximated by an explicit, long bit string (60 bits is far more than enough for floating-point precision).

```
1 def f(omega_bits):
2     """f(omega) = sum_n 2^-n * omega_n, from an explicit (long,
3     truncated) approximation 'omega_bits' of the infinite sequence."""
4     return sum(2 ** -(n + 1) * int(b) for n, b in enumerate(omega_bits))
5
6 zeros, ones = "0" * 60, "1" * 60
7 print("f(0^inf)      =", f(zeros))           # -> 0.0
8 print("f(010^inf)   =", f("01" + zeros))    # -> 0.25
9 print("f(110^inf)   =", f("11" + zeros))    # -> 0.75
10 print("f(10^inf)    =", f("1" + zeros))     # -> 0.5
11 print("f(01^inf)    =", f("0" + ones))      # -> 0.5 (same limit as above!)
12 print("f((01)^inf)  =", f("01" * 30))      # -> 1/3
```

Output matches Example 2.1.9 exactly, including the subtle point that 10^∞ and 01^∞ both map to 0.5.

Python: Cylinder Sets as Intervals

We confirm that the cylinder set Γ_x corresponds to an interval of width exactly $2^{-\ell(x)}$.

```
1 def cylinder_interval(x):
2     """Gamma_x corresponds to [b(x)/2^len(x), (b(x)+1)/2^len(x)]."""
3     if x == "":
4         return (0.0, 1.0)
5     n = len(x)
6     lo = int(x, 2) / 2 ** n
7     return (lo, lo + 2 ** (-n))
8
9 for x in ["", "0", "1", "01", "10", "011"]:
10     lo, hi = cylinder_interval(x)
11     print(f"Gamma_{x or 'eps'} = [{lo:.4f}, {hi:.4f}]   width={hi-lo:.4f}")
```

Output: widths are 1, 0.5, 0.5, 0.25, 0.25, 0.125 respectively — exactly $2^{-\ell(x)}$, confirming Remark 2.1.11.

Section 2.1 Summary

- $\mathbb{B}^* = \bigcup_{n \geq 0} \mathbb{B}^n$ is the set of all finite binary strings; $\mathbb{B}^\infty$ is the set of infinite binary sequences.
- The bijection $\langle \cdot \rangle : \mathbb{B}^* \rightarrow \mathbb{N}_0$, $\langle x \rangle = 2^{\ell(x)} + b(x) - 1$, lets us treat strings and natural numbers interchangeably; $\langle x \rangle$ grows exponentially in $\ell(x)$ (Proposition 2.1.3).
- **Prefix-free** codeword sets make messages uniquely decodable without a delimiter (Theorem 2.1.7). The self-delimiting family E_i trades code length for simplicity: E_0 costs $O(2^{\ell(x)})$ bits, while $\bar{x} = E_1(x)$ and $x' = E_2(x)$ cost only $\ell(x) + O(\log \ell(x))$ bits.
- **Cylinder sets** $\Gamma_x = \{x\omega : \omega \in \mathbb{B}^\infty\}$ give $\mathbb{B}^\infty$ a geometric picture as nested closed intervals of $[0, 1]$ of width $2^{-\ell(x)}$, via the map $f(\omega) = \sum_{n \geq 1} 2^{-n} \omega_n$. This interval structure is the foundation on which Section 2.2 builds probability measures over infinite sequences.